Lab6 – UVM Testbench

This lab's week has an easy description: your task will be to convert a monolithic, SV-only testbench into a modular and flexible UVM testbench. For that, we will use another module which can be found in a modern processor (though not part of the pipeline): a memory controller. A memory controller is a digital circuit that manages the flow of data between a computer's CPU and other components (such as DMA, or I/O devices), and the main memory, ensuring efficient data access and integrity. The design we will use today is quite simple: it will basically arbitrate access to memory from the CPU and the DMA. For simplicity, we will assume that the memory itself is part of the module (normally, it will be another module).

We will provide you with the source code for the design and a complete testbench able to generate stimulus and check correctness of the code. However, this testbench was designed by a newbie verification engineer and implemented it as a single SystemVerilog module, without a modular and reusable structure. Your task will consist of taking this testbench and make a nice, clear, and standardized UVM testbench following what we learn during our course lectures. In this assignment there won't be any specification of the controller module. The different tasks of the sequences, driver, monitor, checker, etc. are already on the provided testbench. You don't need to reinvent them, but to reorganize and readapt to make it UVM compatible.

NOTE: Creating your first UVM testbench can be challenging, with common pitfalls like incorrect instantiations or connections. Be patient and refer to examples, such as the coffee machine from the YouTube video. Your second attempt will likely go more smoothly.

1. Get Ready for this Lab

In the web of the course at https://docencia.ac.upc.edu/master/MIRI/PV/labs.html, for session Lab6, you will find a package called controller.tgz, which contains the design (simple_mem_ctrl.sv) and testbench (tb_simple_mem_ctrl.sv) for the memory controller.

Start by having a deep look at the current testbench code to find the different "expected" parts.

Things to consider:

- Is it worthwhile to have multiple sequences?
- Is it worthwhile to have multiple agents?
- Is it worthwhile to have multiple environments?
- Etc.

2. Design: Controller

The design of the controller is simple: a dual-master memory controller. Master 0 (CPU) has fixed priority over Master 1 (DMA). Synchronous writes on clock, and combinational reads.

For each master's interface, the signals are the following:

Signal	Direction	Description
`addr`	Input	Base address of transaction (word address)

wdata	Input	Write data (valid when we=1)
rdata	Output	Read data (valid when we=0)
we	Input	Write enable (1=write, 0=read)
req	Input	Request valid
gnt	Output	This master has been granted, request to be processed

The arbitration is quite straightforward:

Condition	Behavior
Only one master valid	That master is serviced immediately
Both masters valid	Master 0 is serviced (priority). Master 1 is not granted until Master 0 de-asserts req0
No master valid	Controller idle

Reads: 0 clock cycles (combinational — immediate once address & grant are stable).
Writes: 1 clock cycle (synchronous — update happens on next rising edge when grant & we are true).

3. How to Write a UVM Testbench

Following is a common structure for a UVM testbench

```
`timescale 1ns/1ps
`include "uvm_macros.svh"

package controller_pkg;

    import uvm_pkg::*;

    class controller_transaction extends uvm_sequence_item;
        `uvm_object_utils(controller_transaction)
        …

    class controller_test extends uvm_test;
        `uvm_component_utils(controller_test)
        …

endpackage : controller_pkg

module tb_uvm_simple_mem_ctrl;

    import uvm_pkg::*;
    import controller_pkg::*;

    // Parameters must match DUT
    parameter ADDR_WIDTH = 8;
    parameter DATA_WIDTH = 32;
    …

endmodule : tb_uvm_simple_mem_ctrl
```

4. What to Turn In

Please provide:

(2 pts) Your pre-release after the 2hr lab

(i) (3 pts) A single PDF document with the following:

1. Please indicate how many hours you spent on this lab. This will not affect your grade, but will be helpful for calibrating the workload for the future.
2. (1 pt) List what changes you plan to make to convert one TB into the other.
3. (1 pt) A diagram of your UVM testbench, showing the different components and how they are connected.
4. (1 pt) At least three command lines to launch your UVM testbench, with different arguments.

(ii) (5 pts) Your UVM testbench code (in a single file named tb_uvm_simple_mem_ctrl.sv)